CPEN 315: Computer Organization & Architecture

16 BIT CPU REPORT

GROUP 25

GODFRED SAFO OFORI – 22046644

CHRISTOPHER Y.E MAWULI - 22071976

Abstract

This report presents the design and implementation of a 16-bit CPU using Logisim Evolution. The system integrates core components such as the ALU, registers, memory, datapath, and control unit to execute instructions using the fetch–decode–execute cycle. The CPU successfully performs arithmetic operations and memory interactions, demonstrating a functional processor design.

Introduction

The design of a Central Processing Unit (CPU) is fundamental to understanding computer architecture. This project focuses on building a 16-bit CPU to demonstrate how instructions are processed at the hardware level. The objective is to design a working processor capable of executing a simple instruction set while reinforcing concepts such as datapath design, control signals, and memory interfacing.

Architecture Design

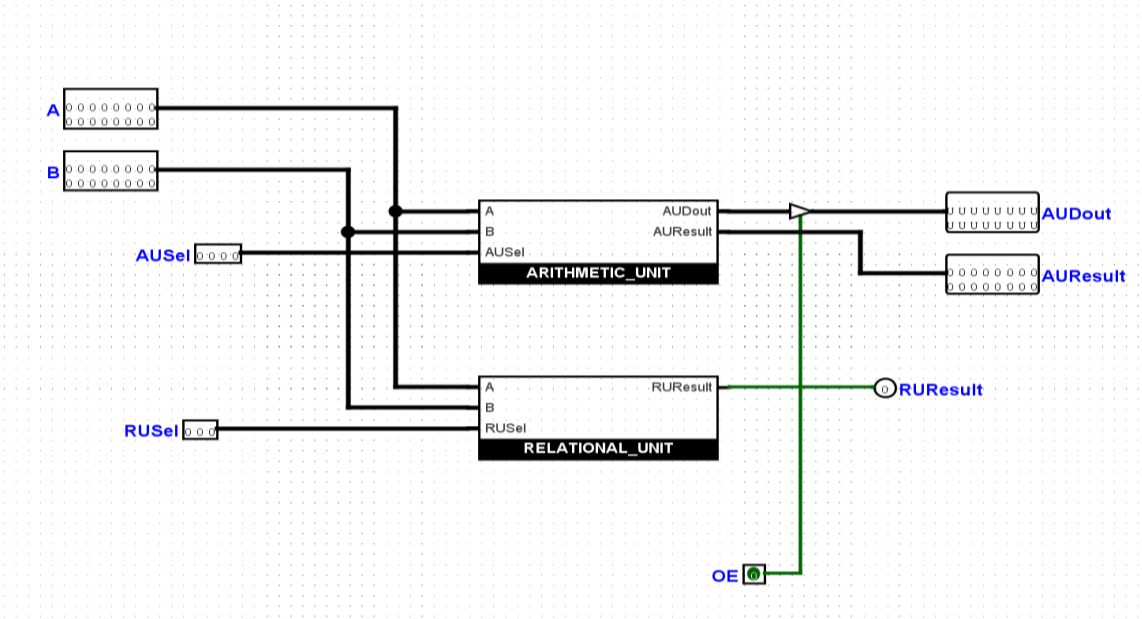
The CPU follows a single-bus Von Neumann architecture. All components share a common data bus. Key components include a 16-bit ALU, registers (A, B, IR, MAR, MBR, OR), RAM, program counter, and control unit. The instruction format consists of 8-bit opcode and 8-bit operand. The system executes four instructions: LoadA, LoadB, Arithmetic, and Store.

Implementation Details

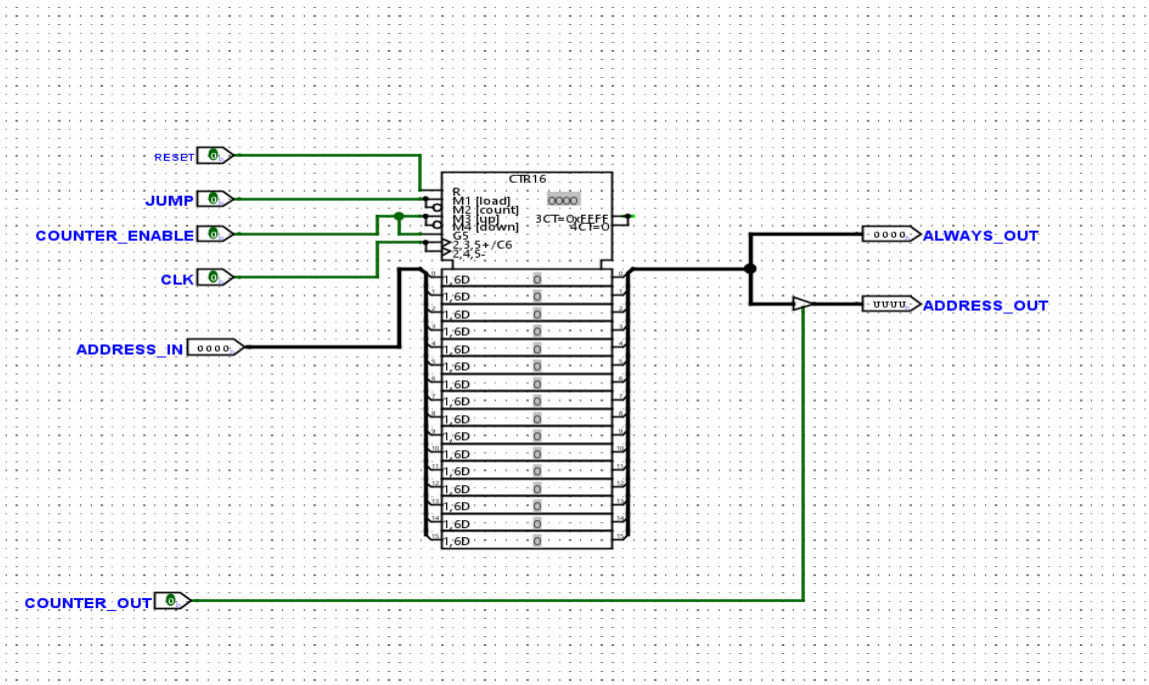
The ALU was implemented using Logisim built-in blocks supporting arithmetic and logical operations. Registers were implemented using memory elements with control signals. The datapath connects all components via a shared bus. The control unit was designed using combinational logic to generate control signals based on opcode and step counter. The program counter cycles through instructions sequentially.

Diagrams

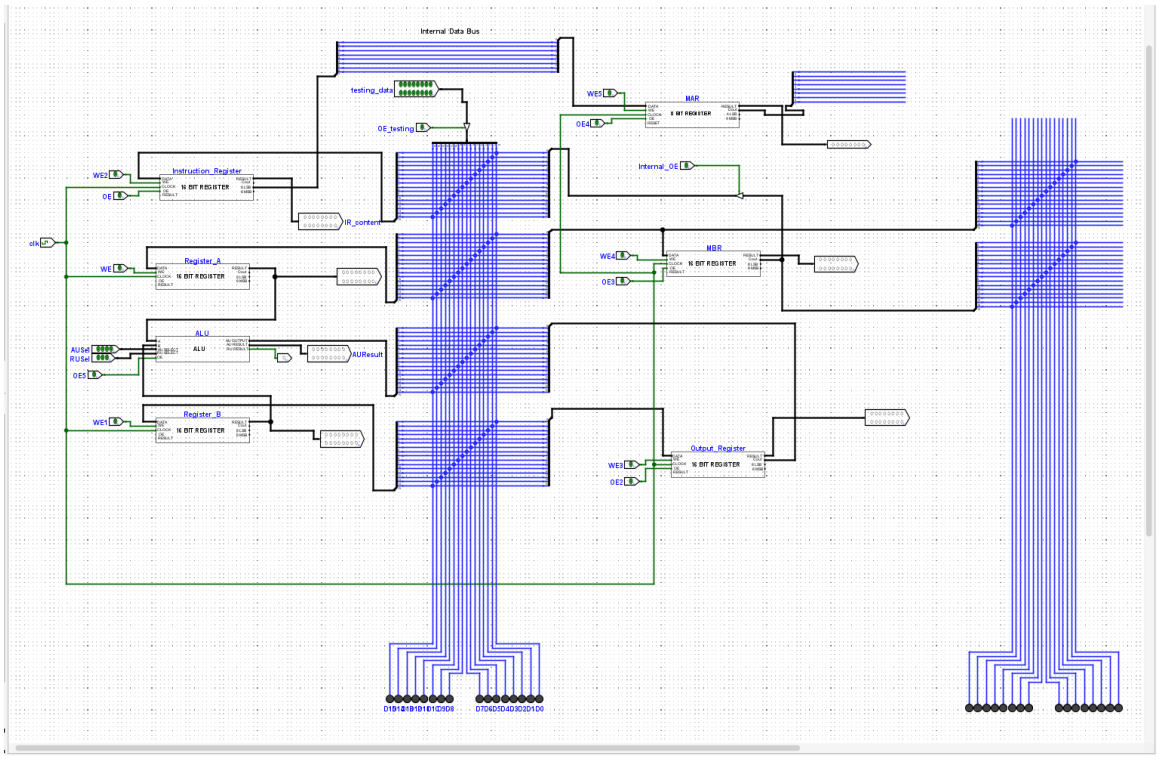
ALU



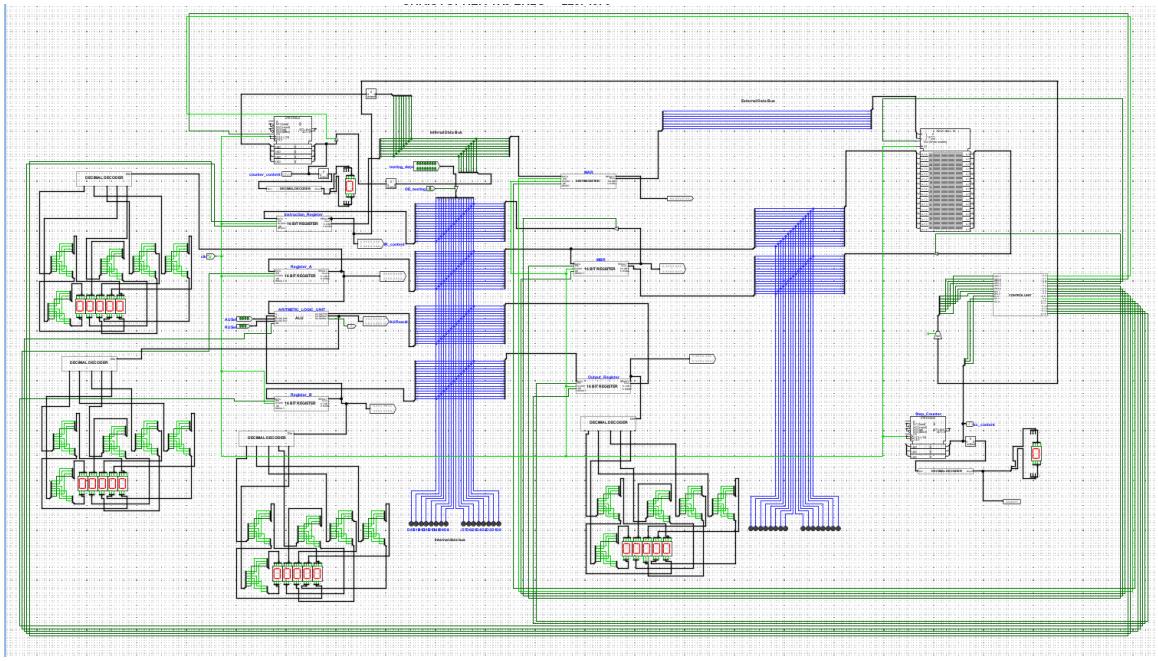
PROGRAM COUNTER



DATAPATH



16 BIT CPU



Expanded Results and Simulation Analysis

Step-by-step execution of the sample program:
Step 1: PC points to first instruction. Instruction is fetched into IR.
Step 2: Control Unit decodes LoadA instruction. MAR loads address, memory value (10) moves to MBR, then to Register A.
Step 3: PC increments. Next instruction fetched.
Step 4: LoadB executed. Value (15) is loaded into Register B.
Step 5: Arithmetic instruction decoded. ALU performs addition (A + B = 25).
Step 6: Result stored in Output Register.
Step 7: Store instruction executed. Result (25) written back to memory.
Throughout simulation, control signals (OE, WE) ensured proper data flow without bus conflicts.
The waveform and state transitions confirm correct implementation of the fetch–decode–execute cycle.

Results and Discussion

The CPU successfully executed a sample program: loading values 10 and 15, performing addition, and storing the result (25). Simulation confirmed correct data flow between registers, ALU operations, and memory updates. The control unit correctly coordinated instruction execution without conflicts.

Challenges and Solutions

Challenges included bus conflicts, incorrect control signal timing, and debugging Logisim errors. These were resolved by careful signal control (OE and WE), structured testing of components, and systematic debugging.

Conclusion and Future Work

The project successfully demonstrates a functional 16-bit CPU. It highlights key principles of processor design. Future improvements include pipelining, expanding the instruction set, and implementing a 32-bit CPU using VHDL.

References

Course lecture notes, Logisim Evolution documentation, and standard computer architecture textbooks.

[1] Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design MIPS Edition: The Hardware/Software Interface* (6th ed.). Morgan Kaufmann.

[2] Harris, D. M., & Harris, S. L. (2021). *Digital Design and Computer Architecture: ARM Edition* (3rd ed.). Morgan Kaufmann.